

lab6

2026-02-25

Q1

a + b

```
## [1] 0.4444445 0.4444444 0.1111111 3.7000000 0.6000000 2.5000000 0.2549510
## [8] 0.3605551 0.0000000
```

```
## [1] 0.4444445 0.4444444 0.1111111 3699.9999753 600.0000017
## [6] 2500.0000000 254.9510311 360.5551318 0.0000000
```

We see that multiplying the data gives us the same parameters. They come in the order p1, p2, p3, mu1, mu2, mu3, sigma1, sigma2, sigma3.

Cosine similarity measures the cos of the angle between our current parameter vector and the previous one. It is independent of a scaling of the mu and sigma. But if we compare different types of parameters in the same cosine similarity, say means hovering around 1000 and fractions between 0-1 then the change in means dominate the cosine similarity. The example below shows this empirically, the cosine similarity changes more if the larger dimension changes by 80% and the smaller by 20% compared to the smaller changing by 80% and the larger dimension changing by 20%.

```
# this example shows
cossim(c(10, 1), c(12, 0.2))
```

```
##          [,1]
## [1,] 0.9965572
```

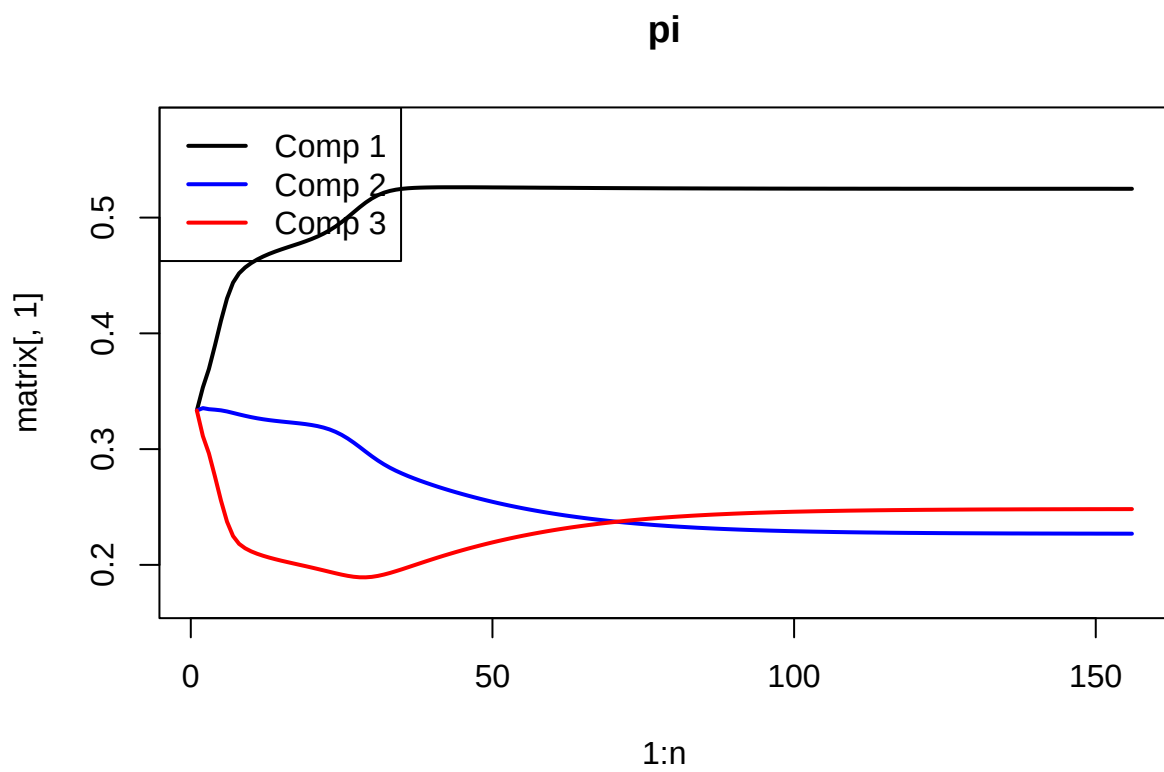
```
cossim(c(10, 1), c(2, 1.2))
```

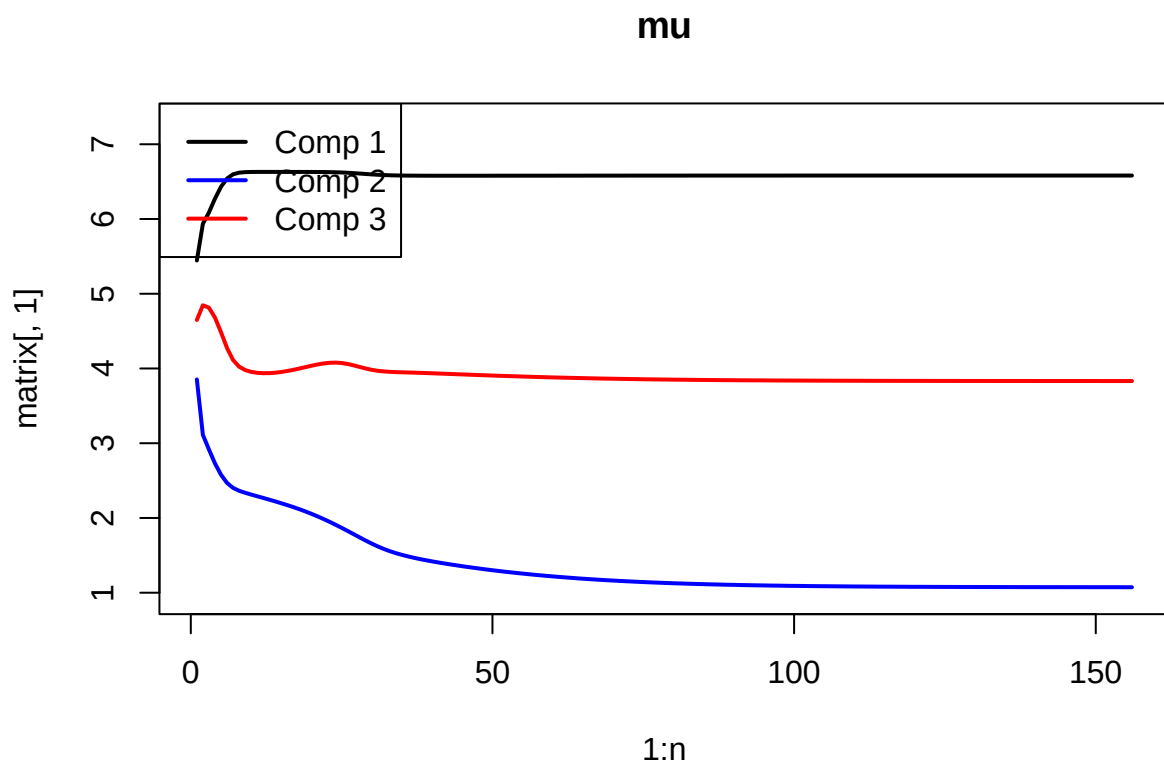
```
##          [,1]
## [1,] 0.9044316
```

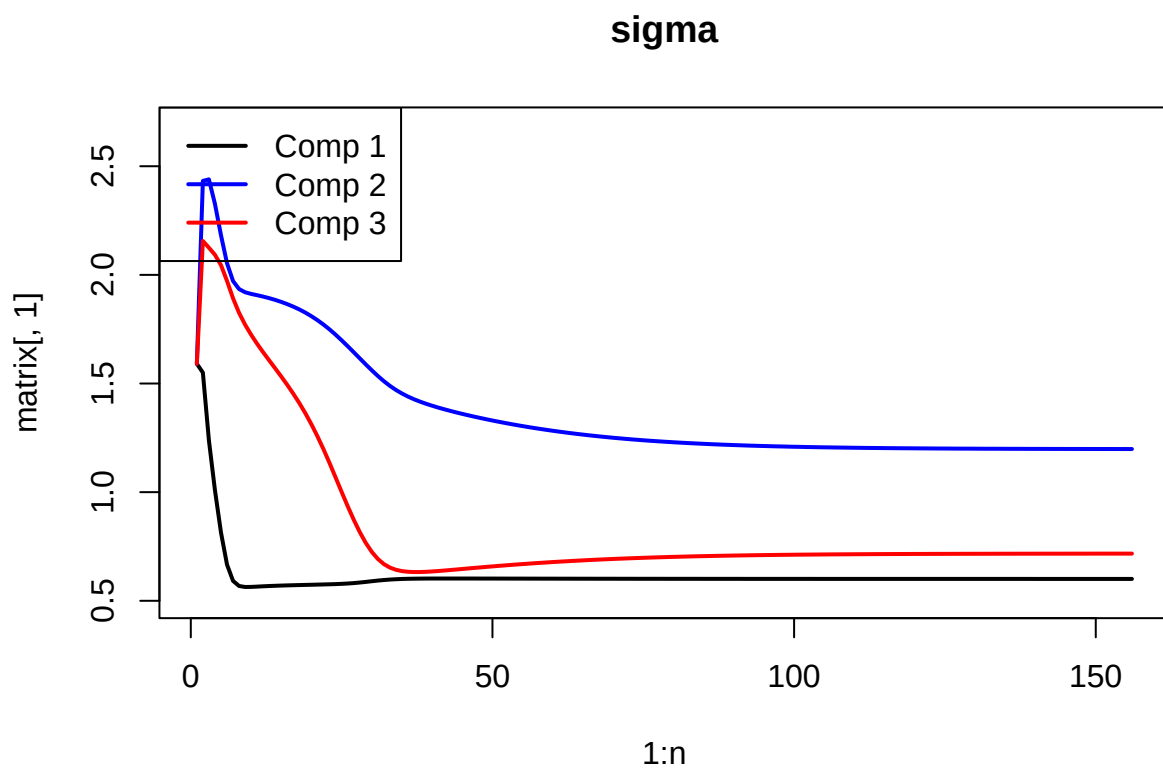
The parameters are on such different scales, using cosine similarity if mu1 changes by 10 it is going to overpower a change of 0.01 in p2. $\cos(v_1, v_2)$ denotes the cosine of the angle between v1 and v2. $1 - \cos$ as $\cos = 1$ when the vecotors are perfectly alligned.

$$cc = \frac{(1 - \cos(v_p, v1_p), (1 - \cos(v_\mu, v1_\mu), (1 - \cos(v_\sigma, v1_\sigma))}{3}$$

```
## [1] 0.5255415 0.2387176 0.2357409 6.5803106 1.1718721 3.8658191 0.6015409
## [8] 1.2558208 0.6901542
```

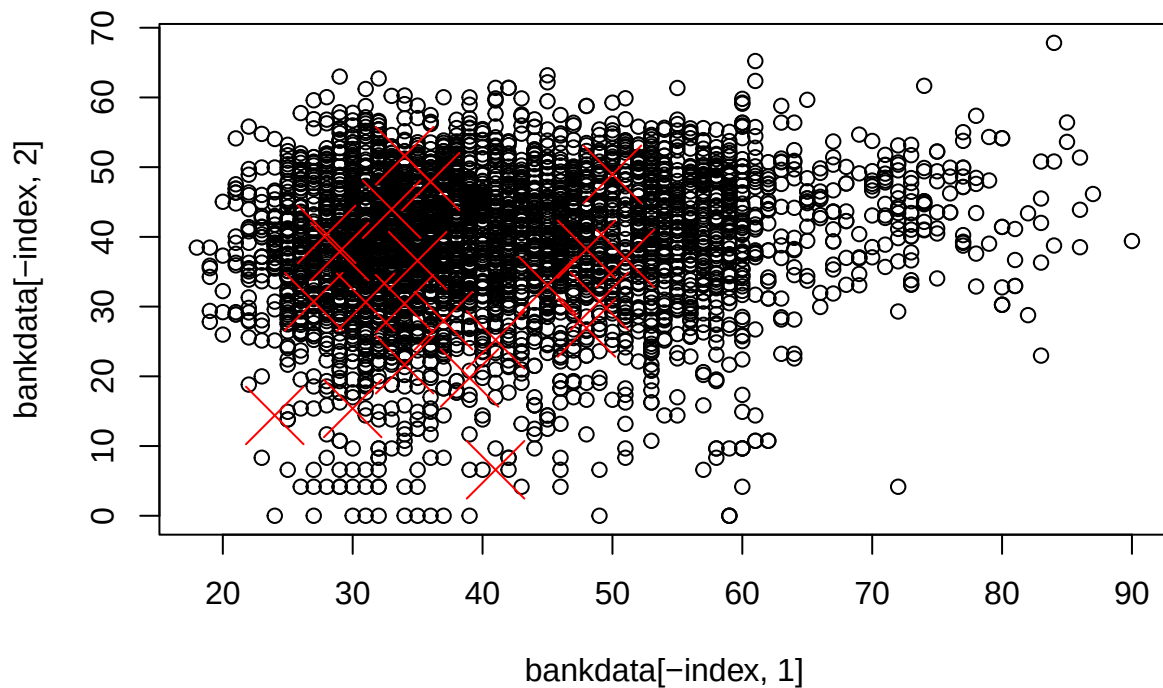






The plots support convergence as they all flatten out. The variance took the longest to converge.

Q2



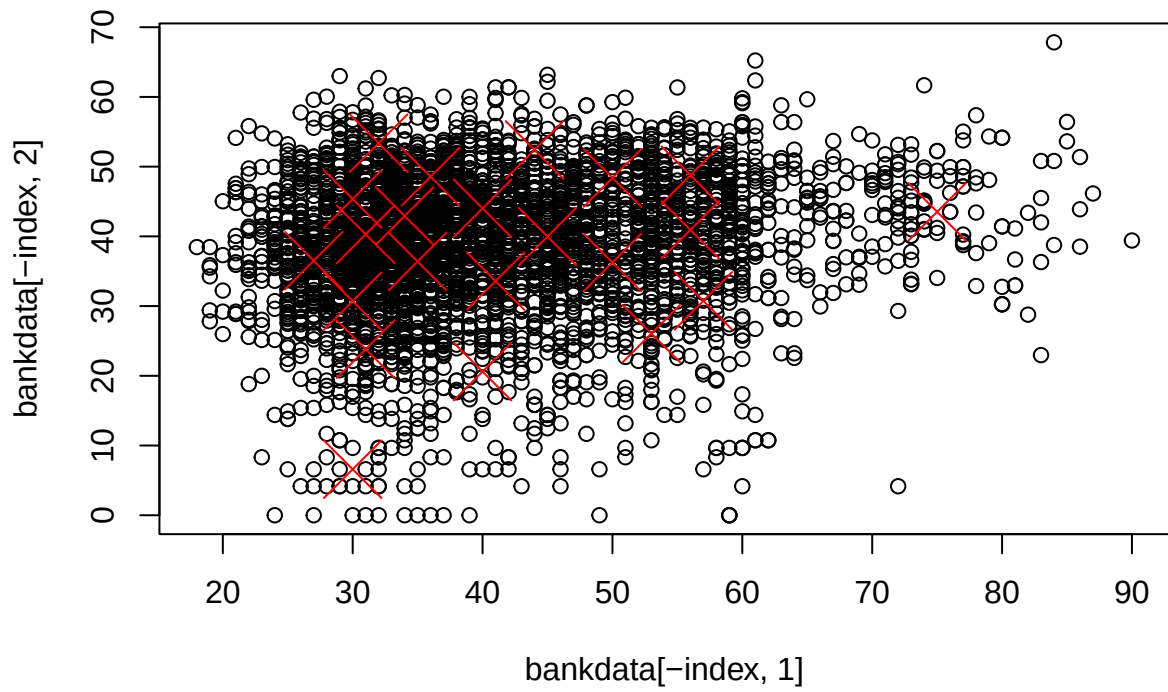
The proposal is a uniform mask, for each individual we swap them out with probability `1-prob_keep`.

`SA(prob_keep = 0.5, alpha = 0.9, beta = 1.02, mj = 50, temp = 10000, it = 200)` This gives 16203 as minimum function value but is painfully slow so cannot run it in the knit.

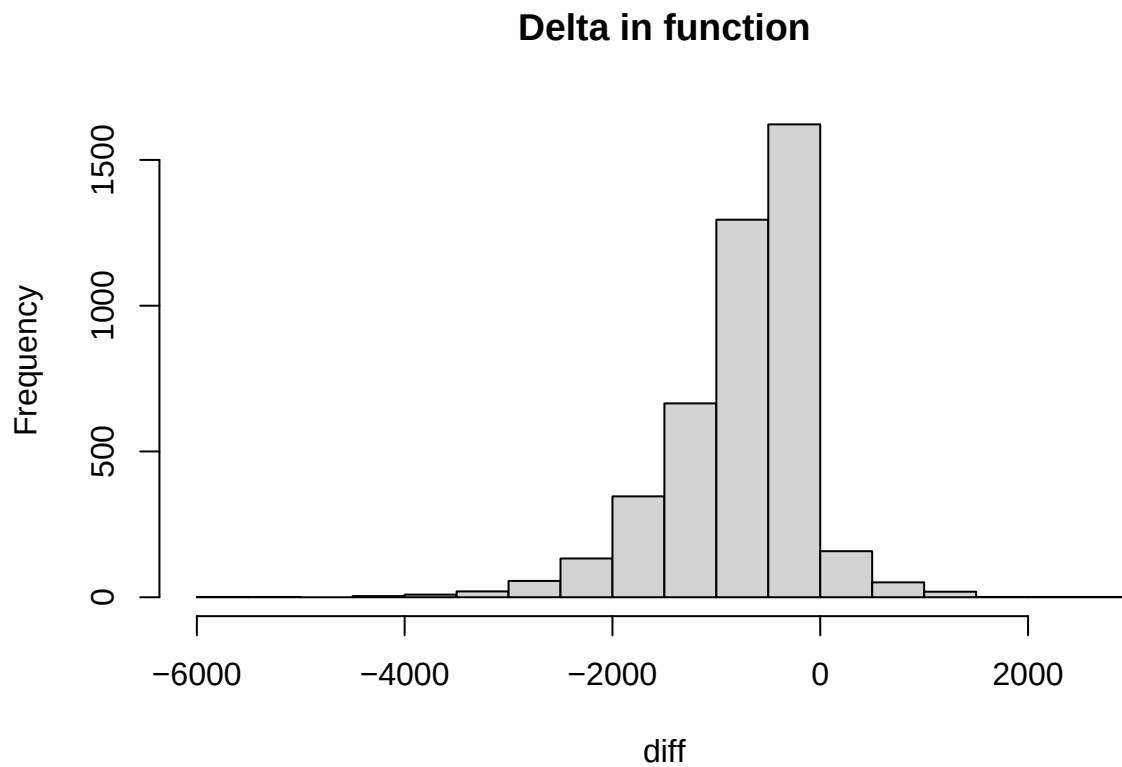
Lower function value and faster, higher starting temperature and lower alpha makes it decay faster, 0.9 probability swaps 2 people on average. Too many iterations with high alpha did not perform as well. The delta plot shows the average function difference and was used to find a good value for the temperature as the scale was unknown. The initial temperature was set such that most values would be accepted to be able to explore more at the start.

```
## [1] 10
## [1] 20
## [1] 30
## [1] 40
## [1] 50
## [1] "Starting function value:"
## [1] 21936.02
```

Best individuals tested



```
## [1] "Best function value:"  
## [1] 15666.21  
## [1] "Average diff:"  
## [1] -725.8411
```

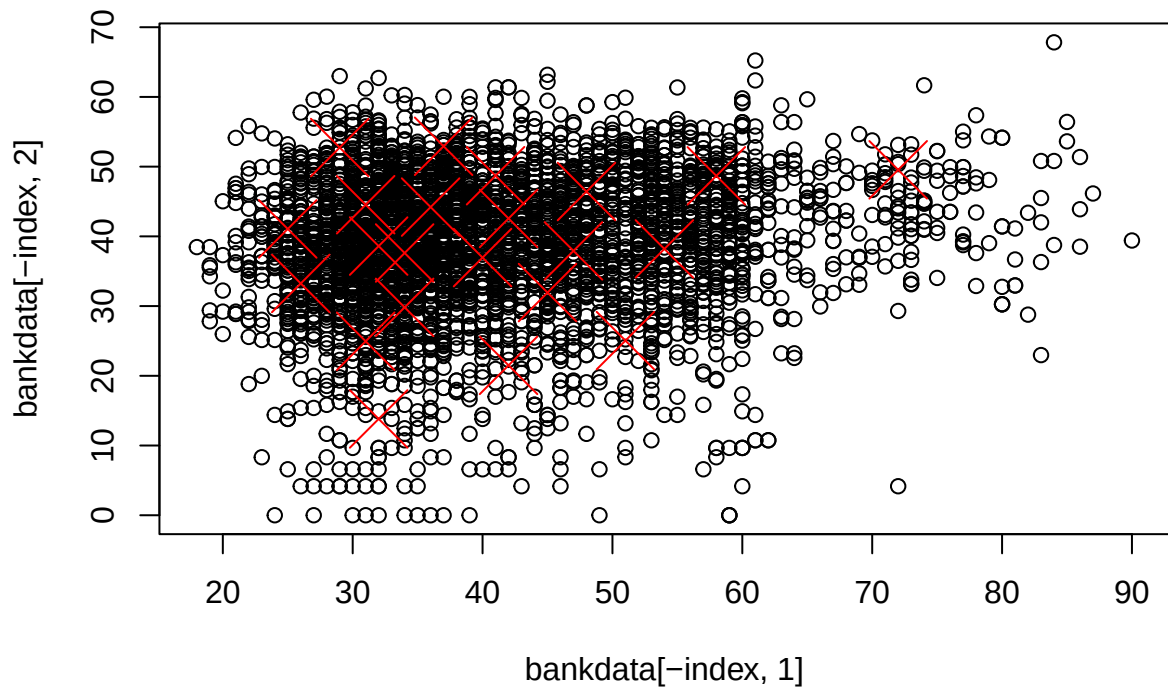


```
## [1] 1747 652 825 917 1580 1556 997 7 521 1514 3048 272 2319 1571 2462
## [16] 212 3954 614 2234 4074 3325 2041
```

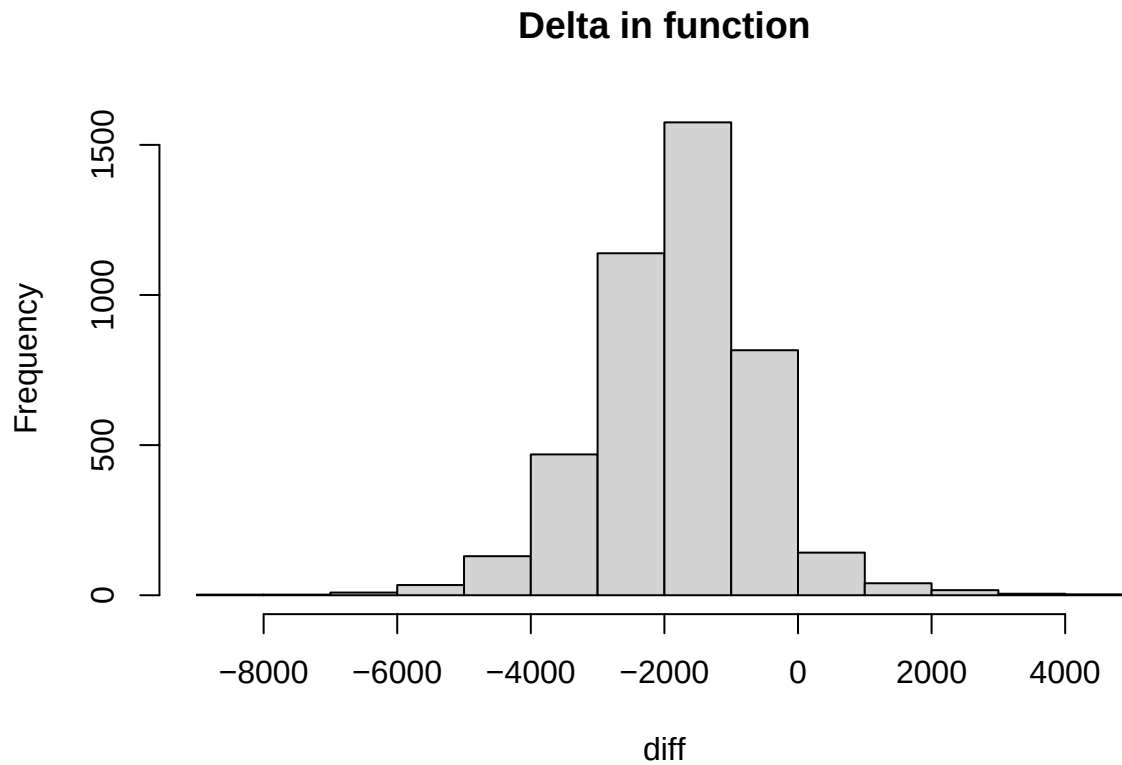
Higher chance of swapping individuals seems to lead to a worse outcome

```
## [1] 10
## [1] 20
## [1] 30
## [1] 40
## [1] 50
## [1] "Starting function value:"
## [1] 21936.02
```

Best individuals tested



```
## [1] "Best function value:"  
## [1] 16281.13  
## [1] "Average diff:"  
## [1] -1811.89
```

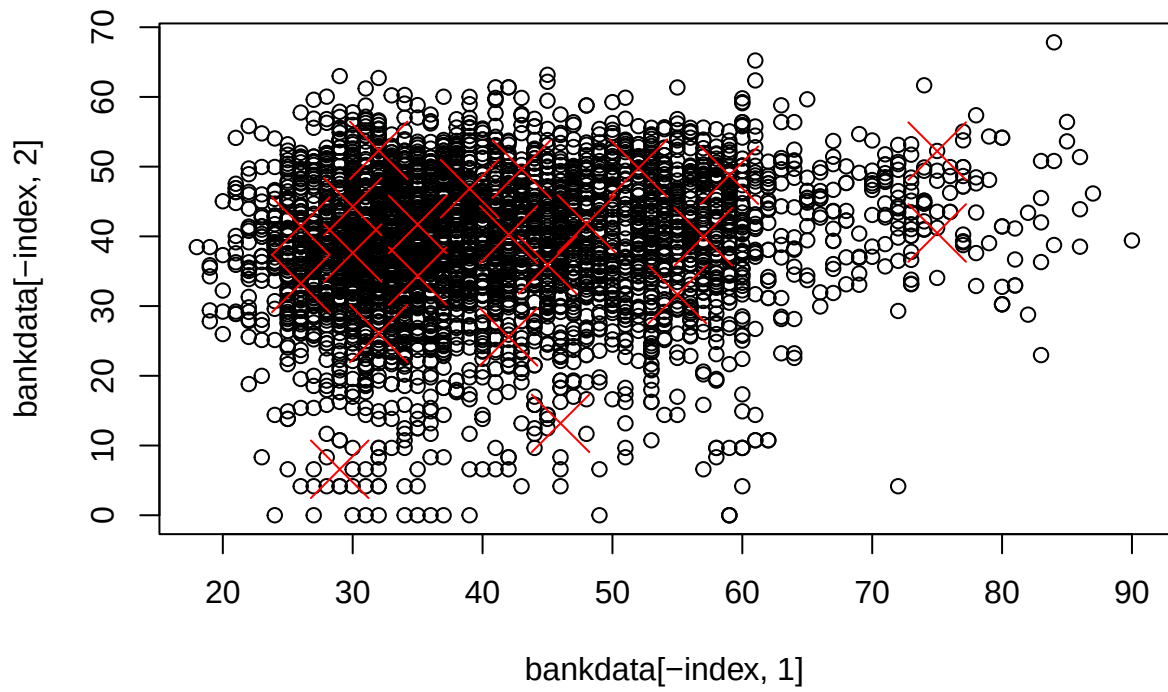



```
## [1] 3206 1315 1761 1647 1666 2103 755 3976 1021 1223 1274 3352 310 2748 1367
## [16] 1445 2718 1491 4017 3491 2383 3707
```

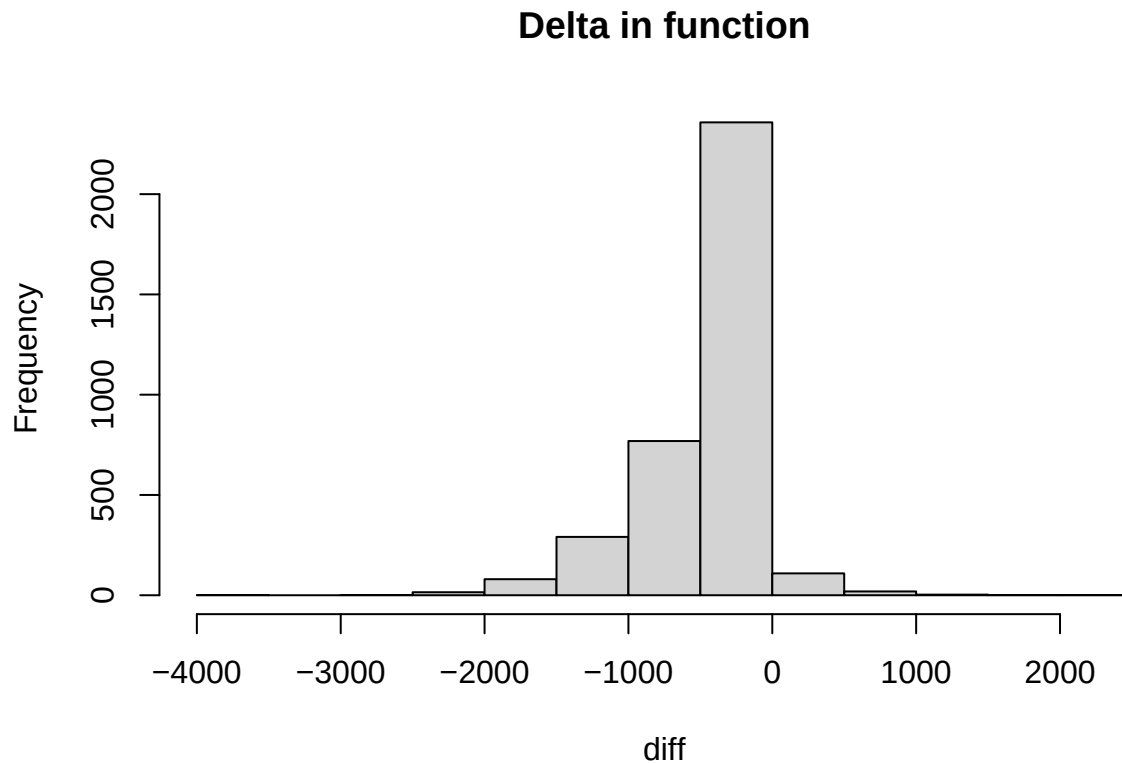
Lower swaprte, on average 1.1 individual performed worse than swapping 2 on average, swapping fewer individuals might lead to locked local minima situations or slower progress.

```
## [1] 10
## [1] 20
## [1] 30
## [1] 40
## [1] 50
## [1] "Starting function value:"
## [1] 21936.02
```

Best individuals tested



```
## [1] "Best function value:"  
## [1] 15721.21  
## [1] "Average diff:"  
## [1] -376.2946
```



```
## [1] 1392 1539 531 3856 1491 1855 2126 4234 257 3776 3402 2359 1028 3903 829
## [16] 3832 110 1537 389 624 487 1803
```

Appendix

```
#####
### Computational statistics, Linköping University, VT2025 ###
### EM algorithm for univariate normal mixtures           ###
### (mixture of two normal distributions)                  ###
### Frank Miller                                           ###
#####

cossim <- function(v1, v2) {
  t(v2)%*%v1 / sqrt(t(v2)%*%v2 * t(v1)%*%v1)
}

emalg <- function(dat, eps=0.000001){

  n      <- length(dat)
  pi_mat <- matrix(NA, nrow=n, ncol=3)
```

```

# define reasonable starting values for parameters; here based on summary statistics for total dataset
p1 <- 1/3
p2 <- 1/3
p3 <- 1/3          # starting value for mixing parameter

sigma1 <- sd(dat)*2/3      # starting value for standard deviation in group taken as 2/3 of total sd
sigma2 <- sigma1
sigma3 <- sigma1

mu1 <- mean(dat) + sigma1/2 # starting values for means, taken a bit lower and a bit higher than overall mean
mu2 <- mean(dat) - sigma1/2
mu3 <- mean(dat)
pv <- c(p1, p2, p3, mu1, mu2, mu3, sigma1, sigma2, sigma3) # parameter vector

cc <- eps + 100          # initialize convergence criterion to avoid stopping the while-loop directly

it <- 2

pi_hist <- matrix(1/3, 1, 3)
mu_hist <- matrix(c(mu1, mu2, mu3), 1, 3)
sigma_hist <- matrix(sigma1, 1, 3)

while (cc>eps){
  pv1 <- pv              # save previous parameter vector
  ### E step ###
  for (j in 1:n){
    w1 <- p1 * dnorm(dat[j], mean=mu1, sd=sigma1)
    w2 <- p2 * dnorm(dat[j], mean=mu2, sd=sigma2)
    w3 <- p3 * dnorm(dat[j], mean=mu3, sd=sigma3)

    sum_w <- w1 + w2 + w3
    pi_mat[j, ] <- c(w1, w2, w3) / sum_w

  }
  ### M step ###
  p1 <- mean(pi_mat[, 1])
  p2 <- mean(pi_mat[, 2])
  p3 <- mean(pi_mat[, 3])

  pi_hist <- rbind(pi_hist, c(p1, p2, p3))

  # Update means
  mu1 <- sum(pi_mat[, 1]* dat)/(p1*n)
  mu2 <- sum(pi_mat[, 2]* dat)/(p2*n)
  mu3 <- sum(pi_mat[, 3]* dat)/(p3*n)
  mu_hist <- rbind(mu_hist, c(mu1, mu2, mu3))

```

```

# Update standard deviations
sigma1 <- sqrt(sum(pi_mat[, 1]*(dat - mu1)^2)/(p1*n))
sigma2 <- sqrt(sum(pi_mat[, 2]*(dat - mu2)^2)/(p2*n))
sigma3 <- sqrt(sum(pi_mat[, 3]*(dat - mu3)^2)/(p3*n))
sigma_hist <- rbind(sigma_hist, c(sigma1, sigma2, sigma3))

#####
pv <- c(p1, p2, p3, mu1, mu2, mu3, sigma1, sigma2, sigma3)

cc      <- mean(1-cossim(pv[1:3], pv1[1:3]), 1-cossim(pv[4:6], pv1[4:6]), 1-cossim(pv[7:9], pv1[7:9])

it <- it + 1
}

return(list(pi_hist = pi_hist, mu_hist = mu_hist, sigma_hist = sigma_hist, pv = pv))
}
# example data and run of the algorithm
data <- c(0.1, 0.5, 0.7, 1.1, 2.5, 3.4, 3.5, 3.9, 4.0)

emalg(data)$pv
emalg(data*1000)$pv

data <- load(file = "threepops.Rdata")

emalg(dat3p)$pv

em <- emalg(dat3p, eps=0.0000000001)

plot_fun <- function(matrix, main) {
  n <- nrow(matrix)
  ylim <- c(min(matrix) * 0.9, max(matrix) * 1.1)

  plot(x = 1:n, y = matrix[, 1], lwd = 2, t = "l", ylim = ylim, main = main)
  lines(x = 1:n,
        y = matrix[, 2],
        col = "blue",
        lwd = 2)
  lines(x = 1:n,
        y = matrix[, 3],
        col = "red",
        lwd = 2)

  legend("topleft", legend = c("Comp 1", "Comp 2", "Comp 3"), col = c("black", "blue", "red"), lwd = 2)
}

plot_fun(em$pi_hist, "pi")
plot_fun(em$mu_hist, "mu")
plot_fun(em$sigma_hist, "sigma")

```

```
#####
### Computational statistics, Linköping University, VT2025 ###
### Criterion function Lab 6, Q2 (space filling design) ###
### Frank Miller ###
###
### We are using in this lab partial data from the ###
### original bankdata available at ###
### https://archive.ics.uci.edu/ml/datasets/Bank+Marketing ###
### See also the publication: ###
### Sérgio Moro, P. Cortez, P. Rita (2014). A data-driven ###
### approach to predict the success of bank telemarketing. ###
### Decision Support Systems. ###
#####

# you need to save the following dataset at the right place and/or add/set the path where it is located
load("bankdata.Rdata")

nclients <- dim(bankdata)[1] # number of individuals in the dataset, here 4364

# criterion function: sum of minimal distance to an element in the subset
# dat is the full dataset (here: bankdata), subs is the set of ids for the subset selected
# subs should be a vector of elements in 1, ..., 4364; for this question, it should be of length 22
# example call: crit(bankdata, 1:22), selecting the first 22 individuals
# result of this function is the criterion to be minimized
crit <- function(dat, subs){
  s <- length(subs)
  dist <- matrix(rep(NA, nclients*s), ncol=s)
  for (i in 1:s){
    dist[, i] <- sqrt((dat[,1]-dat[subs[i],1])^2+(dat[,2]-dat[subs[i],2])^2)
  }
  sum(apply(dist, 1, min))
}

# it is good to identify the individuals in the full set by their id (1, ..., 4364),
# then we can sample from this set for the starting subset:
fullset <- 1:nclients

set.seed(123)
index <- sample(fullset, 22)

plot(bankdata[-index, 1], bankdata[-index, 2])
points(bankdata[index, 1], bankdata[index, 2], col = "red", cex = 4, pch = 4)

SA <- function(prob_keep = 0.5, alpha = 0.8, beta = 0.5, temp = 4000, mj = 10, it = 100) {
  set.seed(123)
  index <- sample(fullset, 22)

  diff <- c()

  it <- it

```

```

x_t <- index

prob_keep <- prob_keep

alpha <- alpha
beta <- beta

mj <- mj
temp <- temp

min_function <- crit(bankdata, index)
min_x <- x_t

function_x_t <- min_function

for (j in 1:it) {
  if (j %>% (round(it/5)) == 0) {
    print(j)
  }
  for (i in 1:mj) {

    swapped_individuals <- prob_keep < runif(22)

    if (sum(swapped_individuals) == 0) {
      x_star <- x_t
    } else {
      not_swapped <- x_t[!swapped_individuals]
      # swapping individuals that got randomly kicked.
      x_star <- c(not_swapped, sample(setdiff(fullset, x_t), sum(swapped_individuals)))
    }

    function_x_star <- crit(bankdata, x_star)
    if (function_x_star < min_function) {
      min_function <- function_x_star
      min_x <- x_star
    }

    delta <- function_x_t - function_x_star
    hx <- exp(delta/temp)
    diff <- c(diff, delta)

    U <- runif(1)
    if (U < hx) {
      x_t <- x_star
      function_x_t <- function_x_star
      #otherwise we do not accept x_t
    }
  }
  temp <- alpha * temp
  mj <- max(1, floor(beta * mj))
}

print("Starting function value:")

```

```

print(crit(bankdata, index))

index <- min_x

plot(bankdata[-index, 1], bankdata[-index, 2], main = "Best individuals tested")
points(bankdata[index, 1], bankdata[index, 2], col = "red", cex = 4, pch = 4)

print("Best function value:")
print(min_function)

print("Average diff:")
print(mean(diff))
hist(diff, main = "Delta in function")

return(min_x)
}

# SA(prob_keep = 0.5, alpha = 0.9, beta = 1.02, mj = 50, temp = 10000, it = 200)
# This gives 16203 as minimum function value but is painfully slow

set.seed(1234)
SA(prob_keep = 0.9, alpha = 0.7, beta = 1.03, mj = 50, temp = 10000, it = 50)

set.seed(1234)
SA(prob_keep = 0.7, alpha = 0.7, beta = 1.03, mj = 50, temp = 10000, it = 50)

set.seed(12345)
SA(prob_keep = 0.95, alpha = 0.7, beta = 1.05, mj = 25, temp = 10000, it = 50)

```